

14

Graphical User Interfaces with Windows Forms: Part I

Objectives

In this chapter you'll:

- Design principles of graphical user interfaces (GUIs).
- Create graphical user interfaces.
- Process events in response to user interactions with GUI controls.
- Understand the namespaces that contain the classes for GUI controls and event handling.
- Create and manipulate various controls.
- Add descriptive **ToolTips** to GUI controls.
- Process mouse and keyboard events.

- 14.1 Introduction
- 14.2 Windows Forms
- 14.3 Event Handling
 - 14.3.1 A Simple Event-Driven GUI
 - 14.3.2 Auto-Generated GUI Code
 - 14.3.3 Delegates and the Event-Handling Mechanism
 - 14.3.4 Another Way to Create Event Handlers
 - 14.3.5 Locating Event Information
- 14.4 Control Properties and Layout
 - 14.4.1 Anchoring and Docking
 - 14.4.2 Using Visual Studio To Edit a GUI's Layout
- 14.5 Labels, TextBoxes and Buttons
- 14.6 GroupBoxes and Panels
- 14.7 CheckBoxes and RadioButtons
 - 14.7.1 CheckBoxes
 - 14.7.2 Combining Font Styles with Bitwise Operators
 - 14.7.3 RadioButtons
- 14.8 PictureBoxes
- 14.9 ToolTips
- 14.10 NumericUpDown Control
- 14.11 Mouse-Event Handling
- 14.12 Keyboard-Event Handling
- 14.13 Wrap-Up

14.1 Introduction

A graphical user interface (GUI) allows a user to interact *visually* with a program. A GUI gives a program a distinctive “look” and “feel.” As an example of a GUI, consider Fig. 14.1, which shows a Visual Studio window containing various GUI controls.



Look-and-Feel Observation 14.1

Consistent user interfaces enable a user to learn new apps more quickly because the apps have the same “look” and “feel.”

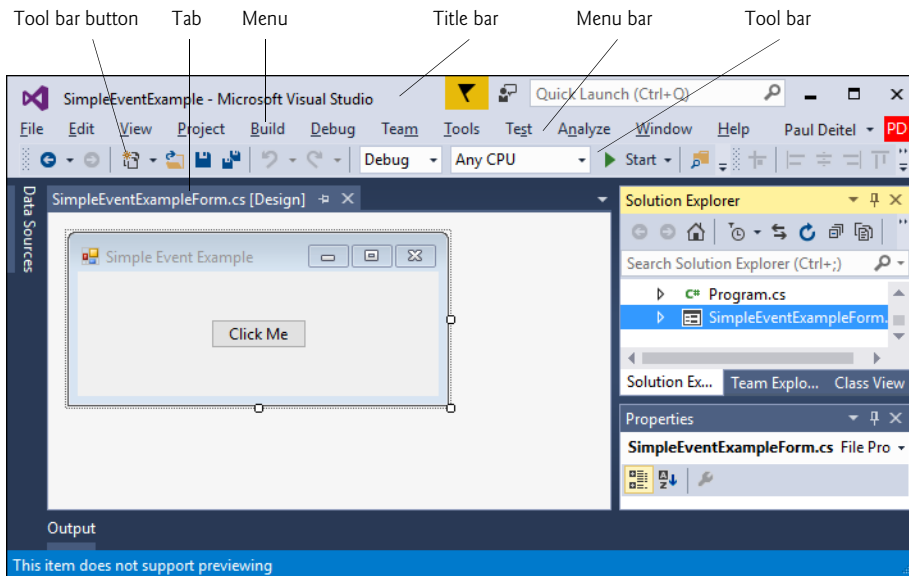


Fig. 14.1 | GUI controls in Visual Studio.

Near the top, there's a *menu bar* containing the menus **File**, **Edit**, **View**, etc. Below that is a *tool bar* of buttons, each with a defined task, such as creating a new project or opening a file. Below that is a *tab* representing a currently open file—this *tabbed view* allows users to switch between the open files. These controls form a user-friendly interface through which you have been interacting with the IDE.

GUIs are built from *GUI controls* (which are sometimes called **components** or **wid-gets**—short for **window gadgets**). GUI controls are *objects* that can display information on the screen or enable users to interact with an app via the mouse, keyboard or some other form of input (such as voice commands). Several common GUI controls are listed in Fig. 14.2—in the sections that follow and in Chapter 15, we discuss each of these in detail. Chapter 15 explores the features and properties of additional GUI controls.

Control	Description
Label	Displays <i>images</i> or <i>uneditable text</i> .
TextBox	Enables the user to <i>enter data via the keyboard</i> . It also can be used to <i>display editable or uneditable text</i> .
Button	Triggers an <i>event</i> when clicked with the mouse.
CheckBox	Specifies an option that can be <i>selected</i> (checked) or <i>unselected</i> (not checked).
ComboBox	Provides a <i>drop-down list</i> of items from which the user can make a <i>selection</i> either by clicking an item in the list or by typing in a box.
ListBox	Provides a <i>list</i> of items from which the user can make a <i>selection</i> by clicking one or more items.
Panel	A <i>container</i> in which controls can be placed and organized.
NumericUpDown	Enables the user to select from a <i>range</i> of numeric input values.

Fig. 14.2 | Some basic GUI controls.

14.2 Windows Forms

Windows Forms is one library that can be used to create GUIs. A *Form* is a graphical element that appears on your computer's desktop; it can be a dialog, a window or an **MDI window** (**multiple document interface window**)—discussed in Chapter 15. A *component* is an instance of a class that implements the **IComponent interface**, which defines the behaviors that components must implement. A *control*, such as a **Button** or **Label**, is a component that has a graphical representation at runtime. Some components lack graphical representations (e.g., class **Timer** of namespace **System.Windows.Forms**—see Chapter 15). Such components are not visible at run time.

Figure 14.3 displays the Windows Forms controls and components from the **C# Toolbox**. The controls and components are organized into categories by functionality. Selecting the category **All Windows Forms** at the top of the **Toolbox** allows you to view all the controls and components from the other tabs in one list (as shown in Fig. 14.3). In this chapter and the next, we discuss many of these controls and components. To add a control or component to a *Form*, select that control or component from the **Toolbox** and drag it onto the *Form*. To *deselect* a control or component, select the **Pointer** item in the **Toolbox** (the icon at the top of the list).

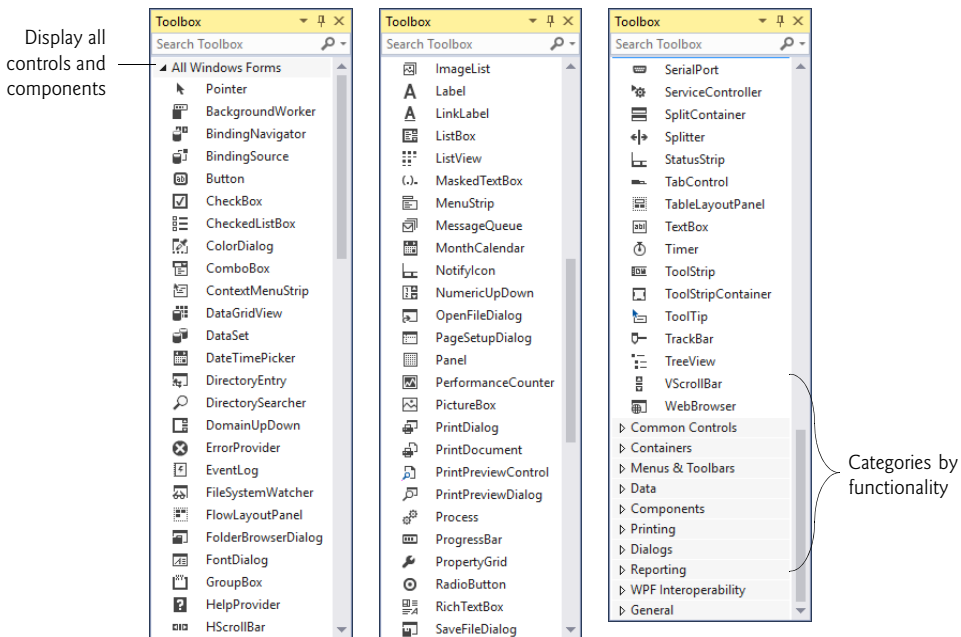


Fig. 14.3 | Components and controls for Windows Forms.

Active Window and Focus

When there are several windows on the screen, the **active window** is the *frontmost* and has a *highlighted title bar*. A window becomes the active window when the user clicks somewhere inside it. The active window is said to “have the **focus**.” For example, in Visual Studio the active window is the **Toolbox** when you’re selecting an item from it, or the **Properties** window when you’re editing a control’s properties.

Auto-Generated Code Stored in Separate File

A Form is a **container** for controls and components. When you drag an item from the **Toolbox** onto the Form, Visual Studio generates code that creates the object and sets its basic properties. This code is updated when the control or component’s properties are modified in the IDE. Removing a control or component from the Form *deletes* the corresponding generated code. The IDE maintains the generated code in a separate file using **partial classes**—classes that are split among multiple files and assembled into a single class by the compiler. You could write this code yourself, but it’s much easier to allow Visual Studio to handle the details. We introduced visual programming concepts in Section 2.6. In this chapter and the next, we use visual programming to build more substantial GUIs.

Common Form Properties, Methods and an Event

Each control or component we present in this chapter is located in namespace `System.Windows.Forms`. To create a Windows Forms app, you generally create a Windows Form, set its properties, add controls to the Form, set their properties and implement *event*

handlers (methods) that respond to *events* generated when a user interacts with the controls. Figure 14.4 lists common Form properties, common methods and a common event.

Form properties, methods and an event	Description
Common Properties	
AcceptButton	Default Button that's clicked when you press <i>Enter</i> .
AutoScroll	<code>bool</code> value (<code>false</code> by default) that allows or disallows <i>scrollbars</i> when needed.
CancelButton	Button that's clicked when the <i>Escape</i> key is pressed.
FormBorderStyle	<i>Border style</i> for the Form (<code>Sizeable</code> by default).
Font	Font of text displayed on the Form, and the <i>default font</i> for controls added to the Form.
Text	Text in the Form's title bar.
Common Methods	
Close	Closes a Form and <i>releases all resources</i> , such as the memory used for the Form's contents. A closed Form cannot be reopened.
Hide	Hides a Form, but does <i>not</i> destroy the Form or release its resources.
Show	Displays a <i>hidden</i> Form.
Common Event	
Load	Occurs before a Form is displayed to the user. You'll learn about events and event-handling in the next section.

Fig. 14.4 | Common Form properties, methods and an event.

14.3 Event Handling

Normally, a user interacts with an app's GUI to indicate the tasks that the app should perform. For example, when you write an e-mail in an e-mail app, clicking the **Send** button tells the app to send the e-mail to the specified e-mail addresses. GUIs are **event driven**. When the user interacts with a GUI component, the interaction—known as an **event**—drives the program to perform a task. Before GUIs, the program told the user what to do next. With GUIs, the user tells the program what to do. Common events (user interactions) that might cause an app to perform a task include

- *clicking* a Button,
- *typing* in a TextBox,
- *selecting* an item from a menu,
- *closing* a window and
- *moving* the mouse.

All GUI controls have events associated with them. Objects of other types also can have associated events as well. A method that performs a task in response to an event is called an **event handler**, and the overall process of responding to events is known as **event handling**.

14.3.1 A Simple Event-Driven GUI

The Form in the app of Fig. 14.5 contains a Button that a user can click to display a `MessageBox`. In line 6, notice the namespace declaration, which is inserted for every class you create—we’ve been removing these from earlier simple examples. Namespaces organize groups of related classes. Recall from Section 7.4.3 that each class’s name is actually a combination of its namespace name, a dot (.) and the class name—again, this is known as the class’s fully qualified name. We’ll use namespaces like this in Chapter 15. If another namespace also contains a class with the same name, the fully qualified class names must be used to distinguish between the classes in the app and prevent a **name conflict** (also called a **name collision**).

```

1  // Fig. 14.5: SimpleEventExampleForm.cs
2  // Simple event handling example.
3  using System;
4  using System.Windows.Forms;
5
6  namespace SimpleEventExample
7  {
8      // Form that shows a simple event handler
9      public partial class SimpleEventExampleForm : Form
10     {
11         // default constructor
12         public SimpleEventExampleForm()
13         {
14             InitializeComponent();
15         }
16
17         // handles click event of Button clickButton
18         private void clickButton_Click(object sender, EventArgs e)
19         {
20             MessageBox.Show("Button was clicked.");
21         }
22     }
23 }
```

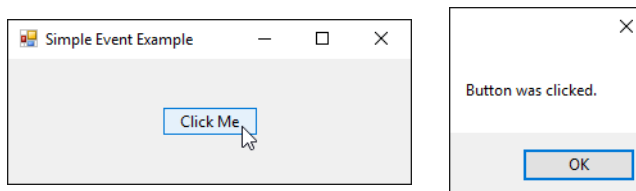


Fig. 14.5 | Simple event-handling example.

Renaming the Form1.cs File

Using the techniques presented in Section 2.6, create a Form containing a Button. First, create a new Windows Forms app names `SimpleEventExample`. Then:

1. Rename the `Form1.cs` file to `SimpleEventExampleForm.cs` in the **Solution Explorer**.

2. Click the Form in the designer, then use the **Properties** window to set the Form's Text property to "Simple Event Example".
3. Set the Form's Font property to Segoe UI, 9pt. To do so, select the Font property in the **Properties** window, then click the *ellipsis (...)* button in the property's value field to display a font dialog.

Adding a Button to the Form

Drag a Button from the **Toolbox** onto the Form. In the **Properties** window, set the (Name) property (which specifies the Button's variable name) to `clickButton` and the Text property to Click Me. By convention, a control's variable name ends with the control's type. For example, in the variable name `clickButton`, "Button" is the control's type.

Adding an Event Handler for the Button's Click Event

When the user clicks the Button in this example, we want the app to respond by displaying a `MessageBox`. To do this, you must create an *event handler* for the Button's *Click event*, which you can do by *double clicking* the Button on the Form. This opens the file containing the following *empty* event handler in the program code:

```
private void clickButton_Click(object sender, EventArgs e)
{
}
```

By convention, the IDE names the event-handler method as *objectName_eventName* (e.g., `clickButton_Click`). The `clickButton_Click` event handler *executes* when the user *clicks* the `clickButton` control.

Event Handler Parameters

Each event handler receives two parameters when it's called. The first—an object reference named `sender` by default—is a reference to the *object that the user interacted with to generate the event*. The second is a reference to an `EventArgs` object (or an object of an `EventArgs` derived class), which is typically named `e`. This object contains additional information about the event that occurred. `EventArgs` is the *base class* of all classes that represent event information.

Displaying a MessageBox

To display a `MessageBox` in response to the event, insert the statement

```
MessageBox.Show("Button was clicked.");
```

in the event handler's body. The resulting event handler appears in lines 18–21 of Fig. 14.5. When you execute the app and click the Button, a `MessageBox` appears displaying the text "Button was clicked."

14.3.2 Auto-Generated GUI Code

Visual Studio places the *auto-generated* GUI code in the Form class's `Designer.cs` file—in this example, the file `SimpleEventExampleForm.Designer.cs`. You can open this file by expanding the Form class's node in the **Solution Explorer** window and double clicking the file name that ends with `Designer.cs`. Figs. 14.6 and 14.7 show this file's contents. The IDE collapses the code in lines 23–57 of Fig. 14.7 by default—you can click the



icon next to line 23 to *expand* the code, then click the



icon next to that line to *collapse* it.

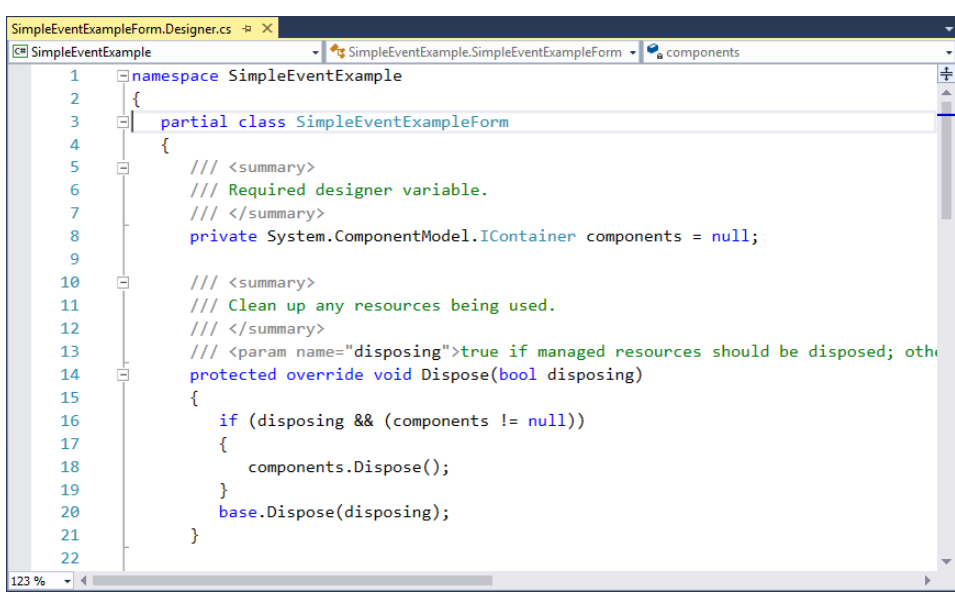
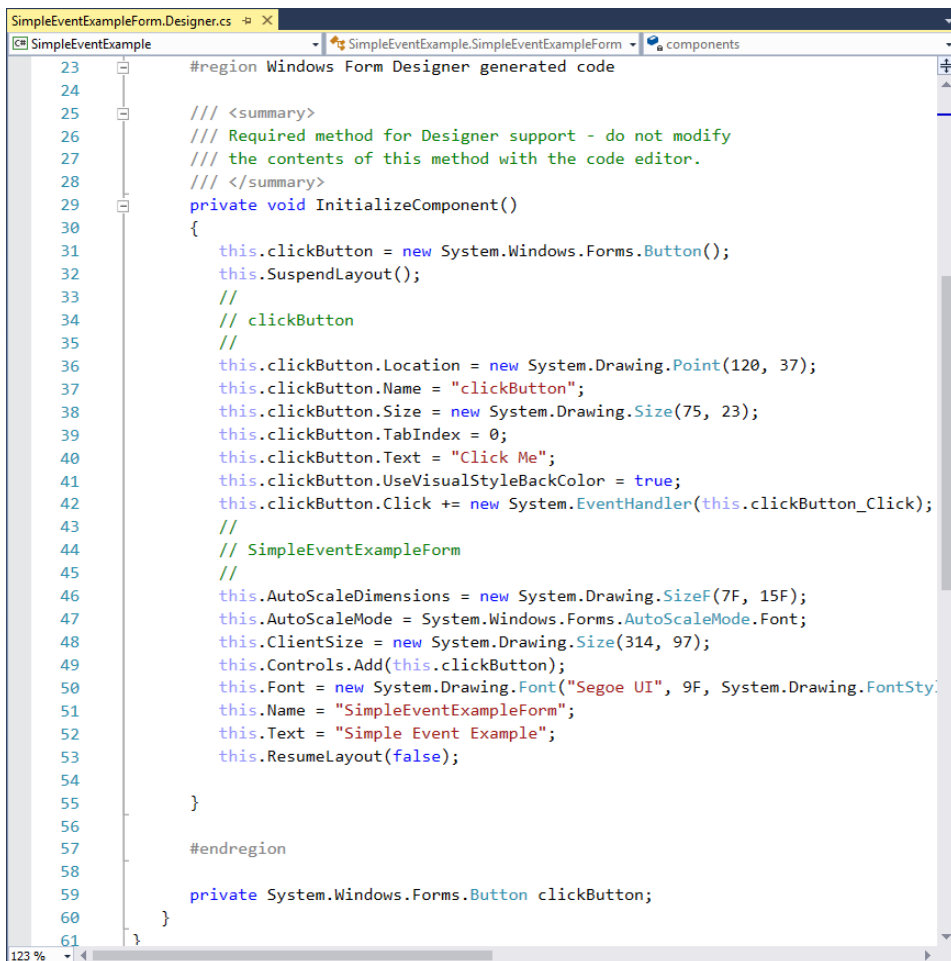


Fig. 14.6 | First half of the Visual Studio generated code file.

Now that you have studied classes and objects in detail, this code will be easier to understand. Since this code is created and maintained by Visual Studio, you generally do not need to look at it. In fact, you do not need to understand most of the code shown here to build GUI apps. However, we now take a closer look to help you understand how GUI apps work.

The auto-generated code that defines the GUI is actually part of the Form's *class*—in this case, `SimpleEventExampleForm`. Line 3 of Fig. 14.6 (and line 9 of Fig. 14.5) uses the `partial` modifier, which allows this class to be split among multiple files, including the files that contain auto-generated code and those in which you write your own code. Line 59 of Fig. 14.7 declares the `clickButton` that we created in **Design** mode. It's declared as an instance variable of class `SimpleEventExampleForm`. By default, all variable declarations for controls created through C#'s *design window* have a `private` access modifier. The code also includes the *Dispose* method for *releasing resources* (Fig. 14.6, lines 14–21) and method *InitializeComponent* (Fig. 14.7, lines 29–55), which contains the code that creates the `Button`, then sets some of the `Button`'s and the `Form`'s properties. The property values correspond to the values set in the **Properties** window for each control. Visual Studio adds comments to the code that it generates, as in lines 33–35. Line 42 was generated when we created the event handler for the `Button`'s `Click` event.



```

23  #region Windows Form Designer generated code
24
25  /// <summary>
26  /// Required method for Designer support - do not modify
27  /// the contents of this method with the code editor.
28  /// </summary>
29  private void InitializeComponent()
30  {
31      this.clickButton = new System.Windows.Forms.Button();
32      this.SuspendLayout();
33      //
34      // clickButton
35      //
36      this.clickButton.Location = new System.Drawing.Point(120, 37);
37      this.clickButton.Name = "clickButton";
38      this.clickButton.Size = new System.Drawing.Size(75, 23);
39      this.clickButton.TabIndex = 0;
40      this.clickButton.Text = "Click Me";
41      this.clickButton.UseVisualStyleBackColor = true;
42      this.clickButton.Click += new System.EventHandler(this.clickButton_Click);
43      //
44      // SimpleEventExampleForm
45      //
46      this.AutoScaleDimensions = new System.Drawing.SizeF(7F, 15F);
47      this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
48      this.ClientSize = new System.Drawing.Size(314, 97);
49      this.Controls.Add(this.clickButton);
50      this.Font = new System.Drawing.Font("Segoe UI", 9F, System.Drawing.FontStyle.Regular, System.Drawing.GraphicsUnit.Point);
51      this.Name = "SimpleEventExampleForm";
52      this.Text = "Simple Event Example";
53      this.ResumeLayout(false);
54
55  }
56
57  #endregion
58
59  private System.Windows.Forms.Button clickButton;
60
61  }

```

Fig. 14.7 | Second half of the Visual Studio generated code file.

Method `InitializeComponent` is called when the Form is created, and establishes such properties as the Form title, the Form size, control sizes and text. Visual Studio also uses the code in this method to create the GUI you see in design view. Changing the code in `InitializeComponent` may prevent Visual Studio from displaying the GUI properly.



Error-Prevention Tip 14.1

*The code in the `Designer.cs` file that's generated by building a GUI in **Design** mode is not meant to be modified directly, which is why this code is placed in a separate file. Modifying this code can prevent the GUI from being displayed correctly in **Design** mode and might cause an app to function incorrectly. In **Design** mode, it's recommended that you modify control properties only in the **Properties** window, not in the `Designer.cs` file.*

14.3.3 Delegates and the Event-Handling Mechanism

The control that generates an event is known as the **event sender**. An event-handling method—known as the *event handler*—responds to a particular event that a control generates. When the event occurs, the event sender calls its event handler to perform a task (i.e., to “handle the event”).

The .NET event-handling mechanism allows you to choose your own names for event-handling methods. However, each event-handling method must declare the proper parameters to receive information about the event that it handles. Since you can choose your own method names, an event sender such as a `Button` cannot know in advance which method will respond to its events. So, we need a mechanism to indicate which method is the event handler for an event.

Delegates

Event handlers are connected to a control’s events via special objects called **delegates**. A delegate type declaration specifies the return type and signature of a method—in event handling, the delegate specifies the return type and arguments for an event handler. GUI controls have predefined delegates that correspond to every event they can generate. For example, the delegate for a `Button`’s `Click` event is of type `EventHandler` (namespace `System`). The online help documentation declares this type as follows:

```
public delegate void EventHandler(object sender, EventArgs e);
```

This uses the **delegate** keyword to declare a delegate type named `EventHandler`, which can hold references to methods that return `void` and receive two parameters—one of type `object` (the *event sender*) and one of type `EventArgs`. If you compare the delegate declaration with `clickButton_Click`’s first line (Fig. 14.5, line 18), you’ll see that this event handler returns the same type and receives the same parameters specified by the `EventHandler` delegate—the parameters’ names need not match. The preceding declaration actually creates an entire class for you. The details of this special class’s declaration are handled by the compiler.

Indicating the Method that a Delegate Should Call

Since each event handler is declared as a delegate, the *event sender* can simply call the appropriate delegate when an event occurs—a `Button` calls the `EventHandler` delegate that corresponds to its `Click` event in response to a click. The delegate’s job is to invoke the appropriate method. To enable the `clickButton_Click` method to be called, Visual Studio assigns `clickButton_Click` to the click `Button`’s `Click` `EventHandler` delegate, as shown in line 42 of Fig. 14.7. This code is added by Visual Studio when you double click the `Button` control in **Design** mode. The expression

```
new System.EventHandler(this.clickButton_Click);
```

creates an `EventHandler` delegate object and initializes it with the `clickButton_Click` method. Line 42 uses the `+=` operator to *add* the delegate to the `Button`’s `Click` `EventHandler` delegate. This enables `clickButton_Click` to respond when a user clicks the `Button`. The `+=` operator is overloaded by the delegate class that’s created by the compiler.

(Optional) Multicast Delegates

You can actually specify that several methods should be invoked in response to one event by adding other delegates to the `Button`’s `Click` event with statements similar to line 42

of Fig. 14.7. Event delegates are **multicast**—they represent a *set of delegate objects* that all have the *same signature*. When an event occurs, the event sender calls *every* method referenced by the multicast delegate. This is known as **event multicasting**. Event delegates derive from class **MulticastDelegate**, which derives from class **Delegate** (both from namespace **System**). For most cases, you'll specify only one event handler for a particular event on a control.

14.3.4 Another Way to Create Event Handlers

For the GUI app in Fig. 14.5, you double clicked the **Button** control on the **Form** to create its event handler. This technique creates an event handler for a control's **default event**—the event that's most frequently used with that control. Controls can generate many different events, and each one can have its own event handler. For instance, your app also can provide an event handler for a **Button**'s **MouseHover** event, which occurs when the mouse pointer remains positioned over the **Button** for a short period of time. We now discuss how to create an event handler for an event that's not a control's default event.

Using the Properties Window to Create Event Handlers

You can create event handlers through the **Properties** window. If you select a control on the **Form**, then click the **Events** icon (the lightning bolt icon in Fig. 14.8) in the **Properties** window, that control's events are listed in the window. You can double click an event's name to display in the editor an existing event handler for that event, or to create the event handler if it does not yet exist in your code. You also can select an event, then use the drop-down list to its right to choose an existing method that should be used as the event handler for that event. The methods that appear in this drop-down list are the **Form** class's methods that have the proper signature to be an event handler for the selected event. You can return to viewing the properties of a control by selecting the **Properties** icon (Fig. 14.8).

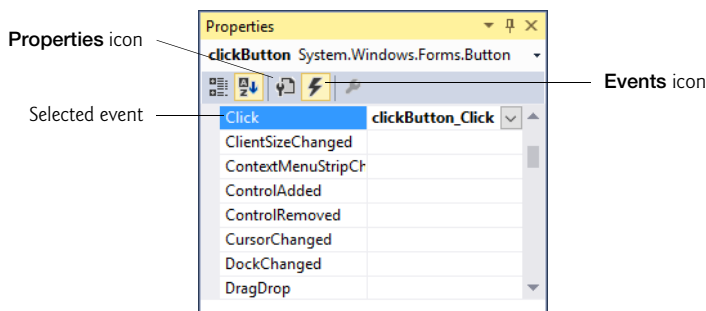


Fig. 14.8 | Viewing events for a **Button** control in the **Properties** window.

A single method can handle events from multiple controls. For example, consider an app that displays **CheckBoxes** that represent bold and italic fonts. The user could select bold, italic or both. In this case, the font's style depends on both **CheckBoxes**, so their events could be handled by the same method, which would determine each **CheckBox**'s state to determine the user's selected font style.

You can specify an event handler for multiple events by selecting multiple controls (drag over them, hold *Shift* and click each or hold *Ctrl* and click each) and selecting a single method in the **Properties** window's **Events** tab. If you create a new event handler this way, you should rename it appropriately so that it does not contain one control's name. You could also select each control individually and specify the same method for each one's event.

14.3.5 Locating Event Information

Read the Visual Studio documentation to learn about the different events raised by each control. To do this, select a control in the IDE (either in your Form's design or in the **Toolbox** and press the *F1* key to display that control's online help (Fig. 14.9).

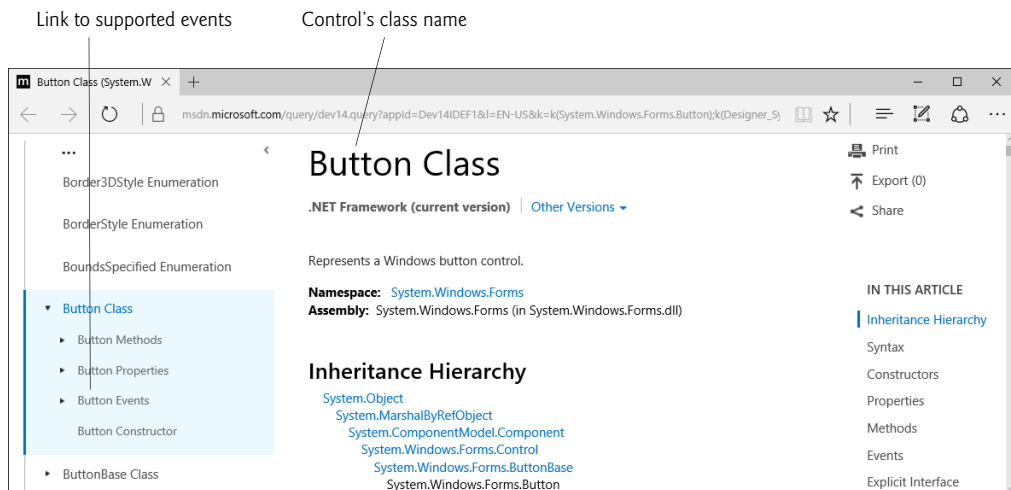


Fig. 14.9 | Link to list of `Button` events.

The web page that's displayed contains basic information about the control's class. In the left column of the page are several links to more information about the class—**Button Methods**, **Button Properties**, **Button Events** and **Button Constructor**. Each link displays a subset of the class's members. Click the link to the list of events for that control (**Button Events** in this case) to display the supported events for that control.

Next, click the name of an event to view its description and examples of its use. We selected the `Click` event to display the information in Fig. 14.10. The `Click` event is a member of class `Control`, an indirect base class of class `Button`. The **Remarks** section of the page discusses the details of the selected event. Alternatively, you could use the **Object Browser** to look up this information in the `System.Windows.Forms` namespace. The **Object Browser** shows only the members *originally* defined in a given class. The `Click` event is originally defined in class `Control` and inherited into `Button`. For this reason, you must look at class `Control` in the **Object Browser** to see the documentation for the `Click` event. See Section 10.11 for more information regarding the **Object Browser**.